

Detecting Misinformation using NER models, Heterogeneous Knowledge Graphs and Graph Neural Networks (GNNs)

David Fujii, Khurram Qureshi, Pablo Aganza

DataSci 266: Natural Language Processing

UC Berkeley School of Information

{[david.fujii](mailto:david.fujii@ischool.berkeley.edu), [khurram](mailto:khurram@ischool.berkeley.edu), [paganza](mailto:paganza@ischool.berkeley.edu)} @ischool.berkeley.edu

Abstract

The rapid proliferation of misinformation on digital platforms poses a significant threat to public discourse, trust in media, and societal well-being. Traditional approaches to fake news detection primarily focus on classifying individual articles as true or false, often ignoring the broader context in which misinformation circulates. This project proposes a more holistic approach by leveraging NLP models to extract entities for knowledge graphs, which look at the complex web of relationships among news articles, social media engagements, publishers, and fact-checking reports. By representing these connections as nodes in a knowledge graph, we aim to uncover whether a piece of news is false and how it influences the information ecosystem.

1. Introduction

Our model employs graph-based learning techniques, particularly graph neural networks (GNNs), to detect and analyze misinformation within a richly connected environment. Using the FakeNewsNet dataset—which includes labeled news articles, metadata on publishers, and fact-checking results—we construct a knowledge graph in which nodes represent key entities such as news articles, claims, and publishers, while edges capture their associations. This graph-based representation enables us to go beyond binary classification and instead assess the relationships of nodes and their correlations with misinformation.

2. Dataset and Project Overview

The inspiration for this project was Marion Meyers, Gerhard Weiss, and Gerasimos Spanakis' use of knowledge graphs in their study "Fake News Detection on Twitter Using Propagation Structures."

Their model, which performed at a 73% accuracy, served as a baseline for our study. Branching off the Marrión, et al. project, our project pipeline includes four main steps to reach our insights: (1) Use Named Entity Recognition (NER) models for extracting entities using spaCy, BERT and RoBERTa, (2) extracting sentiment using Valence Aware Dictionary and sEntiment Reasoner (VADER), (3) knowledge graph construction to model influence and detect falsehoods within the network and (4) the application of graph neural networks (GNNs). At the start of the project, we used Neo4j to store the data into a graph form; however, the processing was made quicker by transforming the data into a Pytorch Geometric data class instead.

Ultimately, our goal is to create a misinformation detection system that is accurate and interpretable—tracing how fake news travels through networks and identifying the key actors, connections and clusters that facilitate its reach.

2.1 Data used

The dataset for training the NER models is from FakeNewsNet (<https://github.com/KaiDMML/FakeNewsNet>). The dataset consists of Twitter posts citing news sources with headlines and includes a fact-check evaluation collected from Politifact and GossipCop in compliance with Twitter's privacy policies. The features captured are the news source, news title, tweet id, and fact-checking label. We chose this dataset because it has been used in many recent studies on misinformation identification in social media and enabled us to compare it with many others (Hu et al., 2025).

2.2 Exploratory Data Analysis (EDA)

As part of exploring the dataset, we wanted to examine the sampling size and distribution of the potential entities used for the GNN.

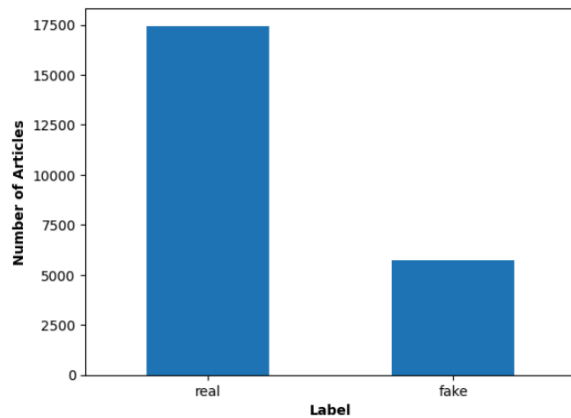


Fig. 1 Fake vs Real News Distribution

Fig. 1 (above) shows how the dataset has significantly more samples of real news than fake news, which we considered when developing the sample data for the models. We addressed the class imbalance by using class weighting prior to training instead of the Marion paper, which used undersampling.

We also looked into the potential entities for the sample data used to train the GNN (shown in Fig. 2 and Fig. 3).

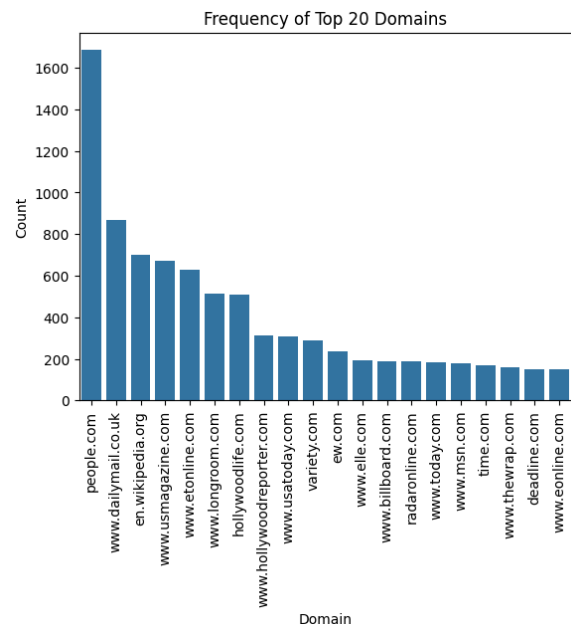


Fig. 2 Top domains in News Titles

Above, we see the distribution of the top domains, which appear to include a wide range of topics

outside the political sphere and are predominantly related to celebrity news, given that people.com is at the top. The appendix includes the top domains for real and fake news, respectively.

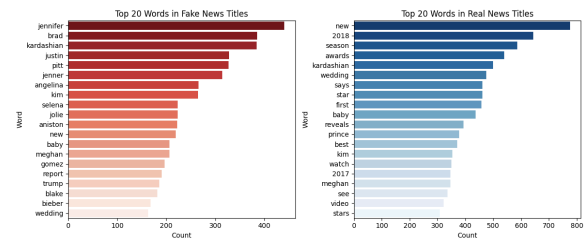


Fig. 3 Top 20 Words (Fake and Real)

To further explore the entities, we derived the top words associated with fake news vs. real news to see what sort of distribution of entities we can expect in the sample data. Consistent with our findings from Fig. 2, we can see that much of the data is related to celebrities. Much of the fake news is associated with persons, and real news seems to include words around a particular subject and/or event.

2.3 Data Pipeline Breakdown

The steps the data takes (pipeline) are as follows:

1. Train NER models using fake news data
2. Produce entities using NER models as nodes for the knowledge graph.
3. Derive sentiment data using VADER and add it as a feature.
4. Transform the data into a knowledge graph
5. Train the GNN model using the data in the graph database.
6. Provide results based on GNN models.

2.4 Project Decisions

Throughout the project, we thought of many different approaches and built code around these methods to detect misinformation, but ultimately, we had to decide against them given the time constraints. One such decision was to use Flair as an NER model. Unfortunately, due to multiple conflicting issues with Google Colab and precise installation package versions, we found that running the code was inconsistent in its ability to run, and the implementation of more complexity further exacerbated the difficulty.

We also looked to use Stellergraph.io, a machine learning graph library, but it proved to be too difficult

because it had dependencies on Python 3.8, which could not be consistently run on Google Colab.

Another decision we decided against was the inclusion of tweet ids to capture users as nodes in our knowledge graph. Despite our efforts, there was no clear way to extract the data using open sources or inexpensive methods. Even if we resorted to using the paid Twitter API, there was a cap on the number of user id retrievals we could obtain daily.

Lastly, as part of our initial idea to use knowledge graphs, we hosted the transformed data into a neo4j database; however, there were limitations on the number of instances we were allowed to host on a free subscription. This caused quite a bit of inconvenience since we could only run one model and variants at a time. Additionally, running the GNN model using the neo4j increased runtime significantly because we would need to load the data into the database and feed the model. Instead, we constructed the knowledge graph directly from the dataframe to allow for concurrent data transformations for all the models and made the ingestion of the data into the GNN model quicker.

3. NLP Methods and NER Models

The models used for this project are spaCy, BERT, and RoBERTa. These models were chosen because they demonstrated the best performances for deriving entities from news headlines. In this study we will compare the evaluation of these NER models paired with the same GNN model. For this project, we produced results with variations for each NER model. The results for each NER model are included in the appendix and usage of heterogeneous or homogeneous graphs. For the purposes of this research, we found that the model performed best when the sentiment feature was included as a feature and when training the GNN using a heterogeneous graph.

3.1 spaCy

The pretrained model we decided to use for spaCy is `en_core_web_lg` (https://huggingface.co/spacy/en_core_web_lg). The transition-based algorithm is great for smaller spans, such as headlines, because it assumes the most relevant words are found close to their initial tokens. The entities that spaCy captures are person, organization, location, NRP (nationalities or religious, political groups), GPE (countries, cities, states), and event, which enables

the GNN to have more features to use relative to BERT and RoBERTa.

3.2 BERT

The pretrained model used for BERT base NER (<https://huggingface.co/dslim/bert-base-NER>) extracts location, organizations, persons, and miscellaneous. It is trained on the CoNLL-2003 Named Entity Recognition dataset. Unlike spaCy, this model does not capture as many entities but is an interesting comparison because the inclusion of more entities could either create better results due to high levels of correlation between the label and added features or provide too much noise.

3.3 RoBERTa

The pretrained NER model we used for RoBERTa was the Jean-Baptiste/roberta-large-ner-english (<https://huggingface.co/Jean-Baptiste/roberta-large-ner-english>). This model was tuned from the robust roberta-large model and, like BERT, is also trained on the CoNLL-2003 Named Entity Recognition dataset. The model can identify persons, organizations, locations, and miscellaneous entities (same as BERT). The result after running the entities that RoBERTa produced into our GNN model was our highest performer at an 84% test accuracy. However, when comparing to spaCy's loss, there is more overfitting with the training data—spaCy results are found in the appendix.

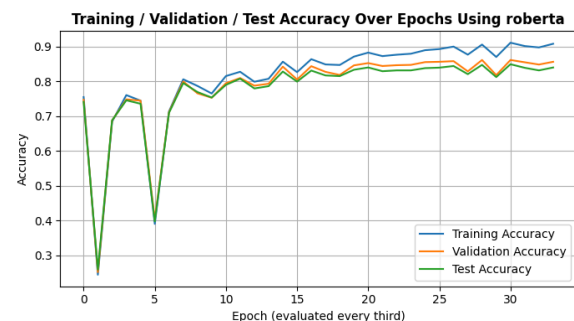
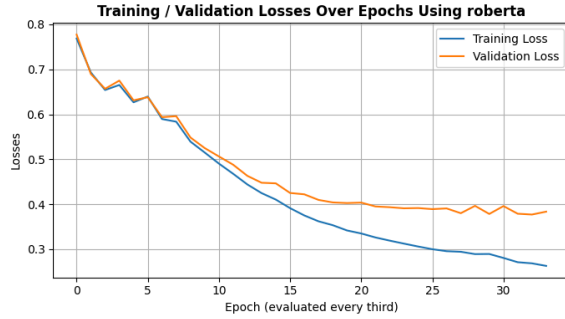


Fig. 4 RoBERTa Accuracy Results (Heterogeneous Graphs)



**Fig. 5 RoBERTa Loss Results
(Homogeneous Graphs)**

3.4 NER model fine-tuning

In addition to optimizing the GNN model using Optuna, we also attempted to fine-tune the NER models on our dataset. Unfortunately, doing so would have required us to have labeling for the entities. Since we didn't have manually annotated entity spans, we couldn't fine-tune the NER models for this project due to time constraints, noting that our project could be used as a springboard for future work in this area.

3.5 VADER

Towards the beginning of this research, we wanted to explore other NLP techniques to improve our existing methodology for using NER. Given the potential relationship between sentiment analysis and misinformation on social media, we came up with the idea to see if sentiment would help enhance our models. We noticed a slight improvement in our models. To accomplish this inclusion, we decided on VADER because it was particularly trained on deriving sentiment from social media.

4 Knowledge Graph Data

After extracting the entities from the corresponding NER models, we transformed them into nodes and their associations into edges. Once the data was generated into a knowledge graph, it was used to feed into the GNN model.

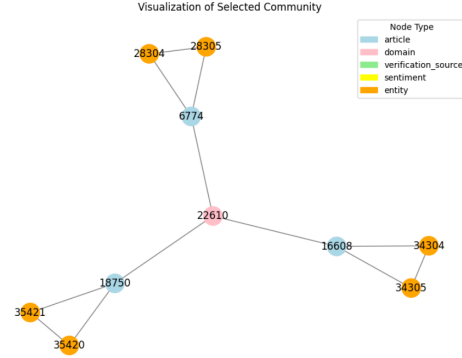


Fig. 6 Knowledge Graph using Louvain's model.

For Fig. 4, we used Louvain's method to identify clusters for each of the communities. There seem to be three main article clusters from a single domain for this particular community. At the start of our project, we transformed the data into homogeneous graphs in which all the nodes were treated as the same type, and relatively good results (found in the appendix). However, we hypothesized that by transforming the data into a heterogeneous graph and adding another layer of complexity for the model, with different types of nodes (entities, articles, domain, sentiment, verification source), we could produce better results, which is what we landed on.

4.1 GNN

GNNs, an extension of RNNs, are a viable way to detect fake news by looking at news sources and other elements to express relationships (Phan, H. T. et al., 2023). By utilizing nodes and edges, the model can utilize these attributes as features. Our method utilizes NER to produce more entities as nodes. Unlike previous studies using knowledge graphs, we can include more types of nodes in the form of entities. To do a fair comparison of the different NER models, we used the same hyperparameter tuning for the GNN model. We used Optuna, a function for tuning hyperparameters to obtain the best results. The hyperparameters used for our model with heterogeneous graphs are shown in Table 1 below. Homogeneous graphs are found in the appendix.

NER Model	Hyperparameter	Value
spaCy	Hidden Channels	49
	Dropout	0.3
	Learning rate	0.00095

	Weight decay	0.0036
BERT	Hidden Channels	96
	Dropout	0.4
	Learning rate	0.00047
	Weight decay	0.00066
RoBERTa	Hidden Channels	69
	Dropout	0.5
	Learning rate	0.00153
	Weight decay	0.00281

Table 1: GNN Hyperparameters (heterogeneous graphs)

5 Comparison With Other Studies

We compared our results with those of similar studies that used knowledge graphs paired with NLPs and/or had the same or similar data.

5.1 Propagation + GDL

The baseline for our research was a model built for misinformation detection using a classification of how the news propagated as features for a Graph Deep Learning (GDL) model. This method ended with a test accuracy of 73%. (Meyers et al.,2020). We greatly improved the accuracy using knowledge graphs with the inclusion of NLP models and switched the knowledge graph machine learning model to GNN.

5.2 Knowledge Graph + RAG

Alternatively, another method used a pipeline in the opposite order to our study. Instead of building graph data using NLP, they trained their NLP model, known as retrieval augmented generation (RAG), to derive output from a knowledge graph with 76% accuracy (Vo, 2024). Although this study didn't include fake news, it was noteworthy to understand different ways to incorporate graph data and NLPs.

5.3 SAFER

Another approach, known as Socially Aware Fake nEws detection fRamework or SAFER, used RoBERTa for text encoding + GNN for community features, including users and articles performed better than our model with their GossipCop dataset. Other accuracy results are found in the appendix.

5.4 Comparisons With Other Methods

Models + Method	Test Accuracy
Our model: spaCy + NER + GNN	83%
Our model: BERT + NER + GNN	81%
Our model: RoBERTa + NER + GNN	84%
Propagation features + GDL (Meyers, et al.,2020)	73%
RAG + Knowledge Graph (Vo, 2024)	76%
SAFER + R-GCN	61% - 94%

Table 2 Test Accuracy Comparisons

More detailed comparison tables are found in the appendix.

6. Conclusion

In this project, we demonstrated a novel approach to misinformation detection by combining NER models, sentiment analysis, and GNNs within a knowledge graph framework. By constructing a heterogeneous knowledge graph that models the relationships among news articles, entities, sentiment, domains, and verification sources, we moved beyond binary classification and towards a more nuanced understanding of misinformation detection. Our experiments showed that integrating sentiment as a feature and leveraging richer entity representations significantly improved performance, with RoBERTa + GNN achieving the highest test accuracy of 84%. While the results with the other models are similar, we hypothesize that RoBERTa's more advanced pretraining and stronger contextual embeddings yield higher-quality entity extraction, which results in a better-structured heterogeneous graph and, consequently, improved GNN performance.

Through comparative analysis with existing methodologies, we validated the effectiveness of our approach and highlighted the value of incorporating diverse NLP signals and graph-based representations. Despite facing technical challenges—such as API limitations, platform incompatibilities, and processing constraints—we

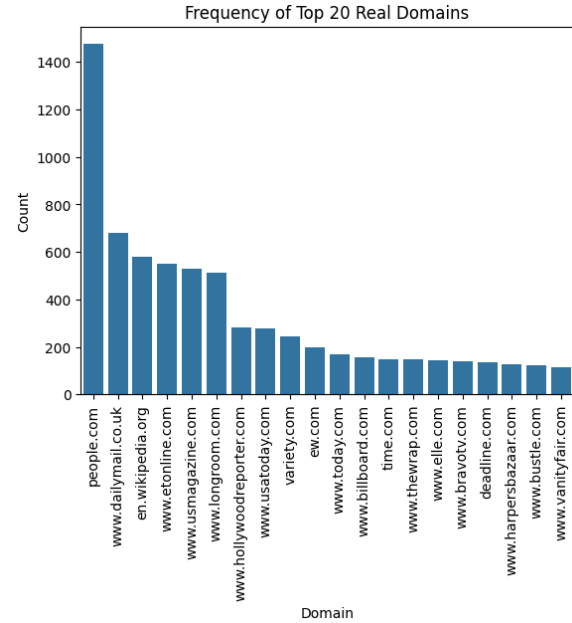
adapted our pipeline for efficiency and scalability using PyTorch Geometric.

If given more time and compute, we would like to focus on expanding to more features, such as Twitter user id, with more nuanced NER models to help combat the complex threat of misinformation.

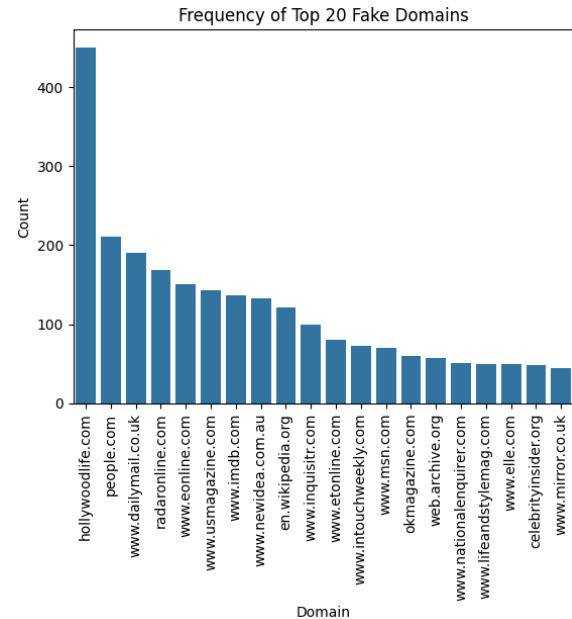
References

- [1] Vo, N. (2024). *Mapping the mind: Knowledge-graph augmented retrieval*. <https://web.stanford.edu/class/cs224n/final-reports/256847497.pdf>
- [2] Phan, H. T., Nguyen, T. T., Nguyen, N. D., & Nguyen, L. T. (2023). *Fake news detection: A survey of graph neural network methods*. *Information Fusion* <https://doi.org/10.1016/j.asoc.2023.110235>
- [3] Hu, B., Zhang, Y., Li, C., & Zhou, J. (2025). *An overview of fake news detection: From a new perspective*. *Journal of Computational Social Science*. <https://www.sciencedirect.com/science/article/pii/S2667325824000414>
- [4] Marion Meyers, Gerhard Weiss, and Gerasimos Spanakis. (2020). *Fake News Detection on Twitter Using Propagation Structures*, https://dke.maastrichtuniversity.nl/jerry.spanakis/wp-content/uploads/2020/11/10.1007@978-3-030-61841-4_MMGWJS.pdf
- [5] S. Chandra, P. Mishra, H. Yannakoudakis, M. Nimishakavi, M. Saeidi, E. Shutova, Graph-based modeling of online communities for fake news detection, 2020, <https://arxiv.org/pdf/2008.06274>

Appendix:



Appendix Fig. 1 Top Real Domains



Appendix Fig. 2 Top Fake Domains

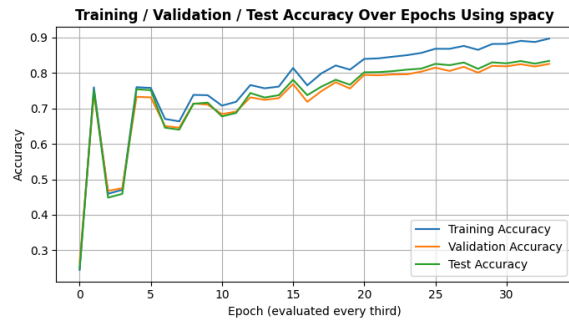
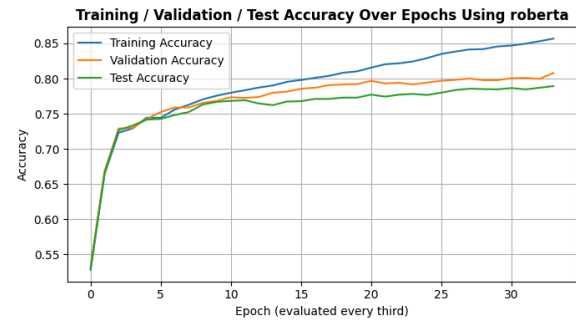


Fig. 3 spaCy Accuracy Results (Homogeneous Graphs)



Appendix Fig. 7 RoBERTa Accuracy Results (Homogeneous Graphs)

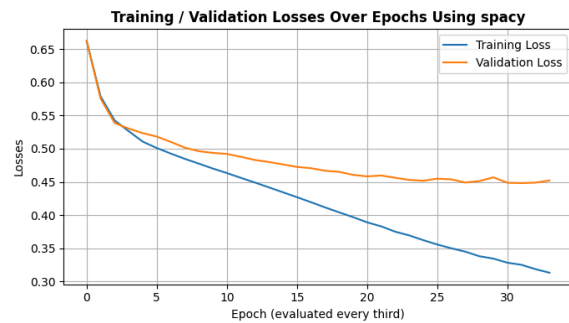
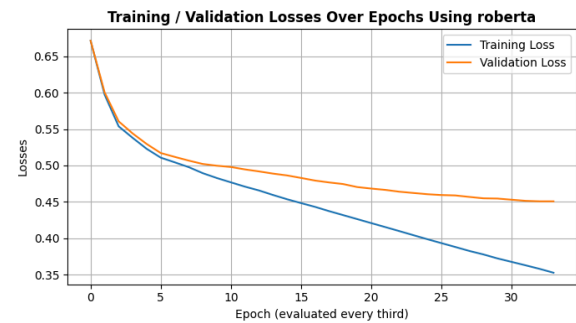
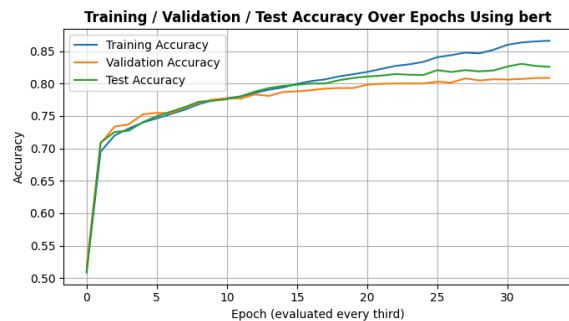


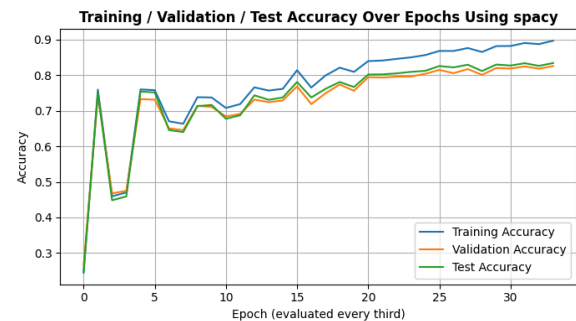
Fig. 4 spaCy Loss Results (Homogeneous Graphs)



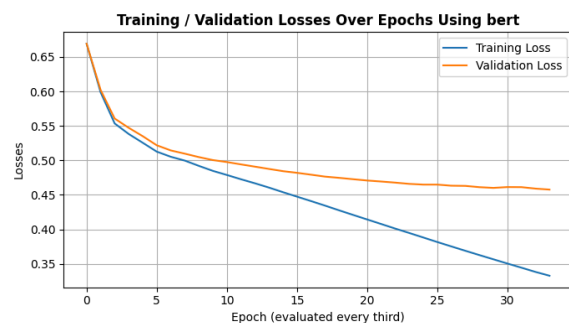
Appendix Fig. 8 RoBERTa Loss Results (Homogeneous Graphs)



Appendix Fig. 5 BERT Accuracy Results (Homogeneous Graphs)



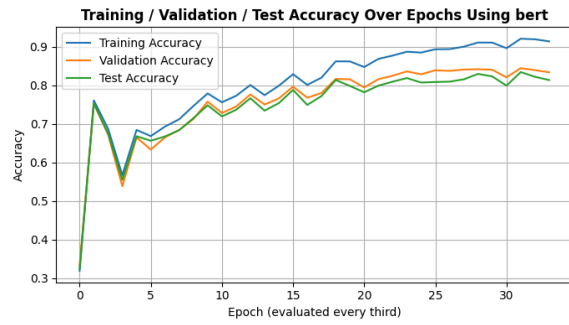
Appendix Fig. 9 spaCy Accuracy Results (Heterogeneous Graphs)



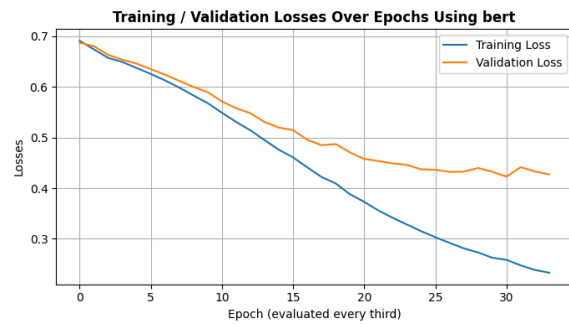
Appendix Fig. 6 BERT Loss Results (Homogeneous Graphs)



Appendix Fig. 10 spaCy Loss Results (Heterogeneous Graphs)



**Appendix Fig. 11 BERT Accuracy Results
(Heterogeneous Graphs)**



**Appendix Fig. 12 BERT Loss Results
(Heterogeneous Graphs)**

NER Model	Hyperparameter	Value
spaCy	Hidden Channels	116
	Dropout	0.4
	Learning rate	0.00583
	Weight decay	0.00030
BERT	Hidden Channels	121
	Dropout	0.5
	Learning rate	0.00484
	Weight decay	0.00011
RoBERTa	Hidden Channels	61
	Dropout	0.4
	Learning rate	0.00671
	Weight decay	0.00019

Appendix Table 1: Homogeneous Graph Tunings

Model	Dataset	F1 (Fake)
Our Model (HeteroGCN + RoBERTa NER)	FakeNewsNet	0.64
SAFER (R-GCN)	GossipCop	0.9469
SAFER (R-GCN)	HealthStory	0.6171
SAFER (R-GAT)	HealthStory	0.6254

Appendix Table 2: SAFER model comparison

Aspect	Notebook GNN (Entity Graph)	Paper GNN (Propagation Graph)
Graph Type	Heterogeneous Knowledge Graph	Retweet Propagation Graph
Node Types	Articles, entities, sentiments, etc.	Tweets and retweets
Node Features	Sentence-BERT embeddings (MiniLM) + type	User stats (followers, favs) + type
Edge Features	None used explicitly	Time difference between tweet & retweet
Learning Task	Node classification (article nodes)	Graph classification (entire news graph)
Architecture	2-layer GCN	GCN + TopK pooling
Accuracy	81–84% (test)	73.3% (GNN), 87% (Random Forest)
Strengths	Rich semantic entity context, pretrained LM	Language-independent, diffusion-based
Weaknesses	No propagation/diffusion signal	No semantic/entity information
Interpretability	Medium (BERT + GNN, but traceable)	High (feature importance in RF)

Appendix Table 3: Propagation + GDL